Load Testing & Performance Analysis Platform

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Yash Bhardwaj [1] Nikhil Khosla [2] Krish Agarwal [3]

August 24, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve software reliability by making application performance testing more accessible, repeatable, and measurable. While the topic applies broadly to web applications and APIs, the immediate engineering objective is to build a deployable platform that allows developers to test how an application behaves under controlled user load before performance problems affect real users.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by first establishing a working backend and automated development pipeline, followed by incremental integration of load generation and performance analysis.
- We will prioritize fast feedback through automated testing and CI-backed builds so that changes can be validated consistently during development.
- The current implementation provides a Flask-based API, automated health testing with pytest, Docker containerization, and GitLab CI-based testing and image building.
- Success is defined by what can be implemented and measured in each iteration, beginning with a reliable service foundation and progressing toward a complete load-testing workflow.

3 Problem Statement

Developers often know that an application functions correctly but may not know how much traffic it can handle before response time degrades or requests begin to fail. Common pain points include:

- **Unknown Capacity:** An application may work correctly for a small number of users, while its maximum sustainable concurrent-user capacity remains unknown.
- **Late Discovery:** Performance bottlenecks may only become visible after deployment, when actual users are already experiencing slow responses or failures.
- **Repeated Manual Work:** Running tests, rebuilding application versions, and redeploying them manually makes performance validation slow and inconsistent.

[1]Roll No: 1024030671, Email: ybhardwaj$_b$e24@*thapar.edu*
[2]Roll No: 1024030672, Email: nkhosla1$_b$e24@*thapar.edu*
[2]Roll No: 1024031155, Email: kagrawal$_b$e24@*thapar.edu*

- **Poor Performance Visibility:** Without structured performance measurements, it is difficult to understand response time, throughput, and failure behavior under different loads.
- **Difficult Version Comparison:** Developers need a consistent way to compare application performance across different load levels and future versions.

Why this matters now (engineering relevance): As applications increasingly serve unpredictable numbers of concurrent users, performance needs to be treated as an engineering property that can be tested and measured rather than assumed.

4 Proposed Solution

. . . Software Proposition

4.1 Overview

Build a web-based Load Testing and Performance Analysis Platform with the following components:

- **Test configuration interface:** Users provide a target URL and configure the desired number of concurrent users, spawn rate, and test duration.
- **Backend API:** A Flask-based API receives and validates test configurations and manages the testing workflow. The current repository already provides the Flask API foundation and health endpoint.
- **Load generation engine:** Locust will be integrated as the planned load-testing engine to generate realistic virtual-user traffic against a user-provided target URL.
- **Performance measurement:** The system will capture metrics such as response time, throughput, request count, and failure rate.
- **Results interface:** Test status and performance results will be presented through a user-facing interface, with historical result comparison planned for later iterations.
- **Automated development and deployment:** GitLab CI, automated testing, Docker containerization, and the existing deployment pipeline will support repeatable delivery.

4.2 Core workflow

1. User provides a target URL and test configuration.
2. Backend validates the configuration and creates a load-test job.
3. The planned Locust engine generates virtual-user traffic against the target application.
4. The system collects response times, throughput, request counts, and failures.
5. Results are processed and presented to the user for performance analysis.
6. Future iterations will support comparison of results across different load levels or application versions.

4.3 Operational constraints for an educational, tier-2/3 setting

- The platform should accept a user-provided target URL rather than being tied to a single application.
- Load testing should be performed only against applications for which the user has authorization to generate traffic.
- The initial implementation will focus on workflow, automation, correctness, and measurable performance rather than novel load-generation algorithms.

- The system should remain deployable through containerization and CI/CD.
- Performance results should be presented using objective metrics rather than subjective observations.

5 Solution Approach

. . . Engineering focus

This is an engineering course project, so we focus on building a reliable and repeatable software system around established load-testing and DevOps technologies rather than developing a novel performance-testing algorithm.

5.1 Duplicate/quality assistance

. . . non-research

Locust will be used as the planned load-generation engine:

- Simulate configurable virtual users against a user-provided target URL.
- Support configurable concurrent users, spawn rate, and test duration.
- Capture standard load-testing measurements such as response time, throughput, request count, and failures.
- Use the generated results to identify the point at which application performance begins to degrade.

The current repository establishes the Flask service foundation, while Locust integration is a planned development stage.

5.2 Web architecture

. . . web-based solution

A modular web architecture:

- **Frontend:** Interface for entering test configuration and viewing test status and performance results.
- **Backend API:** Flask-based REST API responsible for receiving and validating test configurations and coordinating test execution.
- **Task layer:** Planned asynchronous execution layer so long-running load tests do not block the API.
- **Load engine:** Planned Locust component responsible for generating virtual-user traffic.
- **Target application:** Any user-provided web application or API that the user is authorized to test.
- **Results layer:** Stores or processes performance measurements for display and future comparison.

The current Flask implementation already provides `/` and `/health` endpoints, establishing the backend foundation.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically install dependencies and run pytest on code changes.
- Build a repeatable Docker image after successful testing.
- Support consistent packaging and deployment of new versions.
- Reduce repeated manual testing and deployment work.

The current repository already contains automated pytest validation and GitLab CI stages for testing and Docker image building.

6 Evaluation Criterion

. . . measurable and attributable

We define success using measurable metrics tied to the core load-testing workflow.

6.1 Primary evaluation metric

Maximum sustainable load: The highest level of concurrent user traffic that the target application can handle while remaining within an acceptable performance range.

- **Target:** identify the load level at which response time or failure rate begins to show significant degradation.
- **Attribution:** measured directly from load-test results generated during controlled test runs.

6.2 Secondary evaluation metrics

- **Response time:** Measure how long requests take under different levels of concurrent traffic.
- **Throughput:** Measure the number of requests successfully handled over time.
- **Failure rate:** Measure the percentage of requests that fail during a test.
- **Load scalability:** Compare application behavior as concurrent users increase.
- **Version performance:** In later iterations, compare performance between different application versions.
- **CI/CD reliability:** Verify that code changes are automatically tested and successfully packaged through the existing GitLab CI pipeline.

6.3 Pilot validation plan

. . . high-level

- Run controlled tests against an authorized target application using increasing levels of concurrent users.
- Record response time, throughput, request count, and failure rate for each test.
- Identify the load level where performance begins to degrade.
- Repeat tests under comparable configurations to evaluate repeatability.
- In later iterations, compare results from different application versions.

7 Scalability

. . . with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support increasing test workloads by:
   - Separating the frontend, backend, task execution, and load-generation components.
   - Moving long-running load tests to asynchronous background execution.
   - Allowing the load-generation layer to evolve toward multiple workers or distributed execution.
   - Separating test execution from the user-facing API so the API remains responsive while tests run.
2. **Operationally deployable (practically):** The system should support:
   - Containerized application deployment using Docker.
   - Automated testing and image building through GitLab CI.
   - Deployment through the existing container/Kubernetes-oriented infrastructure.
   - Future expansion to multiple load-testing workers as demand increases.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a load-testing and performance-analysis platform:

- Flask for the REST API and backend service.
- Locust for virtual-user simulation and load generation.
- pytest for automated unit and integration-level testing.
- Docker for repeatable application packaging.
- GitLab CI/CD for automated testing and build workflows.
- K3s/Kubernetes for container orchestration and deployment.

We will use these mature components to focus on workflow design, correctness, automation, and measurable performance rather than inventing new infrastructure. The current repository already uses Flask and pytest, while Docker and GitLab CI provide the current containerization and automation foundation.

9 Project scope and deliverables

. . . iteration-friendly

9.1 Initial deliverable

. . . first iteration

- Flask-based backend API with basic health monitoring.
- Automated backend test for the health endpoint.
- Dockerized application.
- GitLab CI pipeline for automated testing and Docker image building.
- Basic API foundation that can be extended with load-test configuration endpoints.
- CD to staging with a single command or pipeline job.

The current repository already provides these foundational components.

9.2 Subsequent deliverables

- Locust-based load-generation engine accepting user-provided target URLs.

- Test configuration for concurrent users, spawn rate, and duration.
- Asynchronous task execution for long-running load tests.
- Performance metrics including response time, throughput, request count, and failure rate.
- Frontend dashboard for configuring tests and viewing results.
- Historical result comparison between different load levels or application versions.

10 Risks and mitigations

- **Target application availability:** Tests depend on the availability of the user-provided target URL. Mitigation: validate connectivity before starting a test and clearly report unreachable targets.
- **Uncontrolled traffic generation:** Excessive load could negatively affect an application. Mitigation: require authorized targets and provide configurable limits on test duration and concurrency.
- **Performance measurement ambiguity:** Different environments can produce different results. Mitigation: record test configuration and use repeatable test conditions when comparing results.
- **Long-running jobs:** Load tests may take significant time and could block the API if executed synchronously. Mitigation: use an asynchronous task layer for test execution.
- **Infrastructure limitations:** A single machine may not generate sufficient load for very large-scale testing. Mitigation: design the load-generation component so it can later support multiple workers or distributed execution.
- **Deployment failures:** Automated deployment can fail because of configuration or container issues. Mitigation: run automated tests before building and deploying new versions.
- **Incomplete feature integration:** The current repository contains the backend, testing, Docker, and CI foundation while Locust, the frontend, metrics, and result comparison are planned. Mitigation: implement these features incrementally and validate each stage before integration.

11 Summary

This project proposes a deployable Load Testing and Performance Analysis Platform that allows developers to evaluate application performance under controlled traffic before real users are affected. It meets the course criteria by emphasizing measurable engineering outcomes, rapid incremental development, automated testing, containerization, and CI/CD.

The current implementation establishes a reliable foundation using Flask, pytest, Docker, and GitLab CI, while subsequent iterations will integrate Locust, asynchronous test execution, performance metrics, a user-facing dashboard, and result comparison. The central engineering objective is simple: *Test application capacity before deployment, measure performance objectively, and make the process repeatable and automated.*